Background notes

Supporting material for understanding the paper. Not presented.

Quantization in one example (illustrative numbers). Weights $x = (0.9, -0.3, 0.1, 0.6)$, $k = 2$ bits, integers $\{0, 1, 2, 3\}$.

1. Grid: smallest value $\to 0$, largest $\to 3$: $s_f = \dfrac{0.9 - (-0.3)}{3} = 0.4$, $b_f = -0.3$.
2. Rescale: $f(x) = (x + 0.3)/0.4 = (3,\ 0,\ 1,\ 2.25)$.
3. Round: $q = (3,\ 0,\ 1,\ 2)$. Stored: 2 bits per weight, plus s_f, b_f.
4. Back: $\hat{x} = 0.4\,q - 0.3 = (0.9, -0.3, 0.1, 0.5)$. The last weight moved $0.6 \to 0.5$: the rounding error $s_f \delta$.

Ridge instead maps q back to $(0.935, -0.285, 0.122, 0.528)$: the error is spread over all four weights, total squared error $0.010 \to 0.007$.

1 Chain rule

Setup. Let $h : \mathbb{R}^n \to \mathbb{R}^m$ be one layer and $\ell : \mathbb{R}^m \to \mathbb{R}$ be everything after it (the rest of the network and the loss), so the loss is $L(x) = \ell\big(h(x)\big)$. The *local Jacobian* of the layer is the $m \times n$ matrix

$$J_h(x) = \frac{\partial h}{\partial x}, \qquad (J_h)_{ij} = \frac{\partial h_i}{\partial x_j}.$$

Chain rule.

$$\nabla_x L \;=\; J_h(x)^\top \, \nabla \ell\big(h(x)\big).$$

Here $\nabla \ell\big(h(x)\big)$ is the *downstream gradient* (what the rest of the network wants the layer's output to do), and $J_h(x)^\top$ converts it into a gradient on the layer's input x.

Intuition. The downstream gradient says how the loss would change if the layer's *output* moved. The local Jacobian says how the layer's output moves when its *input* x moves. Multiplying the two gives how the loss changes when x moves, which is what gradient descent needs to update x. Backpropagation repeats this layer by layer, so every layer's local Jacobian is a factor in the gradient that reaches earlier layers.

Two cases relevant to this paper.

1. **Local Jacobian is zero**, $J_h(x) = 0$. Then

   $$\nabla_x L = 0 \quad \text{for every downstream gradient.}$$

   Meaning: small changes to x do not change the layer's output, so the loss gives x no direction to move. Gradient descent never updates x, and nothing before this layer learns either. This is what rounding (quantization) does.

2. **Local Jacobian is the identity**, $J_h(x) = I$. Then

$$\nabla_x L = \nabla \ell\big(h(x)\big).$$

Meaning: the downstream gradient is passed back unchanged, as if the layer did nothing. Whatever the layer actually did to x (for example, adding a rounding error) has no effect on the gradient, so earlier layers get no signal about it. This is what the Straight-Through Estimator (STE) assumes.

2 Ridge regression

Why needed. The paper's dequantizer g is a ridge-regression fit of the original x from the rounded q (with $u = q$, $v = x$ below).

Notation. For $u, v \in \mathbb{R}^N$: $\bar{u} = \frac{1}{N}\sum_i u_i$, $\overline{uv} = \frac{1}{N}\sum_i u_i v_i$, $\mathrm{Var}_u = \overline{u^2} - \bar{u}^2$, $\mathrm{Cov}_{uv} = \overline{uv} - \bar{u}\,\bar{v}$, and $\mathbf{1}$ is the all-ones vector.

Problem. Fit $v \approx a\,u + c\mathbf{1}$, penalizing only the slope:

$$\min_{a,c} \ \frac{1}{2N}\|a\,u + c\mathbf{1} - v\|^2 + \frac{\lambda}{2}a^2, \qquad \lambda > 0.$$

Setting derivatives to zero:

$$c^* = \bar{v} - a^*\bar{u}, \qquad a^* = \frac{\mathrm{Cov}_{uv}}{\mathrm{Var}_u + \lambda}.$$

Without intercept ($c = 0$): $a^* = \dfrac{\overline{uv}}{\overline{u^2} + \lambda}$.

Fit. The vector of the line's predictions at the data points:

$$\text{fit} = a^* u + c^*\mathbf{1} = a^*(u - \bar{u}\mathbf{1}) + \bar{v}\mathbf{1}.$$

Role of λ. (i) *Stability:* the denominator is at least $\lambda > 0$, so no division by zero even if all u_i are equal. (ii) *Shrinkage:* as $\lambda \to \infty$, $a^* \to 0$ and the fit becomes the mean $\bar{v}\mathbf{1}$.

3 Quantization

In short. We select a grid of integers, assign one integer to each of our real numbers (rescale, then round to the nearest), and then try to get our real numbers back from those integers. Only the integers (plus one s_f, b_f per block) are stored; the recovery is approximate, and the gap is the rounding error. The paper keeps the first two steps and changes only the last one: how to get back from the integers to the best possible real numbers.

Setup. A block $x \in \mathbb{R}^N$ (weights or activations) is mapped to a k-bit integer grid $\{q_{\min}, \dots, q_{\max}\}$ with 2^k levels. Notation AaWw: a-bit activations, w-bit weights.

Step 1: prequantization f. Rescale x onto the grid's range.

- Linear (centred data, e.g. weights): $f(x) = \dfrac{x}{s_f}, \quad s_f = \dfrac{\max_i |x_i|}{q_{\max}}$.
- Affine (uncentred data, e.g. ReLU activations): $f(x) = \dfrac{x - b_f\mathbf{1}}{s_f}, \quad s_f = \dfrac{x_{\max} - x_{\min}}{q_{\max} - q_{\min}}, \quad b_f = x_{\min} - s_f\, q_{\min}$.

Step 2: rounding and the error. round is nearest integer, entrywise.

$$q = \text{round}\big(f(x)\big) = f(x) + \delta, \qquad \delta = \text{round}\big(f(x)\big) - f(x), \qquad |\delta_i| \le \tfrac{1}{2}.$$

Step 3: standard dequantization. Undo the rescaling ($b_f = 0$ in the linear case):

$$\hat{x} = s_f\, q + b_f \mathbf{1} = x + s_f\, \delta.$$

Quantization is therefore x plus an additive error of size at most $s_f/2$ per entry; fewer bits means larger s_f, hence larger error.

4 The problem: zero gradient and the Straight-Through Estimator

The chain during training. The network stores the real x; the next layer uses the quantized-then-dequantized $\hat{x}$:

$$x \xrightarrow{f} f(x) \xrightarrow{\text{round}} q \xrightarrow{s_f q + b_f \mathbf{1}} \hat{x} \longrightarrow \text{rest of network} \longrightarrow L,$$

where

- $f(x) = \dfrac{x - b_f \mathbf{1}}{s_f}$ rescales x onto the integer grid ($b_f = 0$ in the linear case); s_f is the step size of the grid in real units, b_f the real value of integer 0;
- $q = \text{round}\big(f(x)\big) = f(x) + \delta$ is the stored integer vector, δ the rounding error, $|\delta_i| \le \frac{1}{2}$;
- $\hat{x} = s_f\, q + b_f \mathbf{1} = x + s_f\, \delta$ is x as seen after quantization.

Chain rule. Gradient descent needs $\nabla_x L$:

$$\nabla_x L = \Big(\frac{\partial \hat{x}}{\partial x}\Big)^{\top} \nabla_{\hat{x}} L, \qquad \frac{\partial \hat{x}}{\partial x} = \underbrace{s_f}_{\text{scale back}} \cdot \underbrace{\frac{\partial q}{\partial f(x)}}_{\text{round}} \cdot \underbrace{\frac{1}{s_f}}_{f} = \frac{\partial q}{\partial f(x)}.$$

The two scalings cancel, so the local gradient is exactly the slope of round.

Zero gradient. round is a staircase: $\frac{d}{du}\,\mathrm{round}(u) = 0$ for $u \notin \mathbb{Z} + \frac{1}{2}$ (undefined at half-integers). Hence $\frac{\partial q}{\partial f(x)} = 0$ almost everywhere and $\nabla_x L = 0$: x and all earlier layers never update (case 1 of Section 1).

STE (Bengio et al., 2013). Treat the rounding error as a constant in the backward pass:

$$q = f(x) + \delta, \qquad \delta = \mathrm{stopgrad}\big(\mathrm{round}(f(x)) - f(x)\big).$$

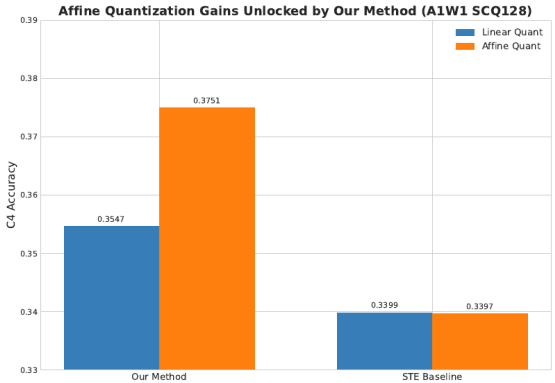
Forward: $q = \mathrm{round}(f(x))$ exactly. Backward: $\frac{\partial q}{\partial f(x)} = I$, so $\frac{\partial \hat{x}}{\partial x} = I$ and $\nabla_x L = \nabla_{\hat{x}} L$ (case 2 of Section 1).

The blind spot.

$$\text{forward:}\quad \hat{x} = x + s_f\,\delta, \qquad\qquad \text{backward:}\quad \frac{\partial \hat{x}}{\partial x} = I.$$

The forward pass is changed by the error, but the local Jacobian is the same as with no quantization at all; δ does not appear in it. Earlier layers get no signal about how their rounding error hurts the loss, so they cannot learn to be robust to it. At 1–2 bits $s_f\delta$ is large and STE training becomes unstable. (δ still affects *where* $\nabla_{\hat{x}} L$ is evaluated; it is absent only from the local factor $\partial\hat{x}/\partial x$, which is the factor the paper changes.)

5 Affine vs. linear at 1 bit (paper Fig. 1b)



Setup. A1W1 (1-bit weights and activations), SCQ128 (one block = 128 weights), metric = next-token accuracy on C4.

What the figure shows.

- STE: linear 0.3399, affine 0.3397. Affine gives nothing.
- Ridge: linear 0.3547, affine 0.3751. Affine gives +0.020.
- Even ridge-linear beats STE-affine.

Why at 1 bit. One bit means two levels. Linear gives two levels symmetric about zero, $\{-s, +s\}$: one free number. Affine gives two levels $\{b,\, b+s\}$: two free numbers, so both levels can sit where the weights actually are. Affine is strictly more expressive; the question is whether the method uses it.

STE does not use it. The levels are s_f, b_f, fixed by $x_{\min}$, $x_{\max}$ before rounding. At 1 bit every weight is reconstructed as either $x_{\min}$ or $x_{\max}$, the two most extreme values, and the gradient carries no information about how well the levels fit (δ is invisible). The extra offset is set by the extremes, not by the data.

Ridge does. $g(q) = s_g q + b_g \mathbf{1}$ is a least-squares fit of x on q. With $q \in \{0, 1\}$ (and $\lambda \to 0$) the fitted levels are

$$g(0) = \text{mean of the weights rounded to 0}, \qquad g(1) = \text{mean of the weights rounded to 1},$$

the two cluster means: the best possible two levels for the current assignment. Linear ridge has only s_g, so its levels are forced to be symmetric; affine ridge places each level independently. And because s_g, b_g depend on q, the gradient tells earlier layers how the assignment affects the fit.

One line. Affine adds a free offset; only a dequantizer that is fitted to the data (ridge) can put that offset to use.

W e have two kinds of quantization: linear, which is just scaling, and affine, which is scaling plus shifting. Because of the shift, affine has more flexibility. At 1 bit this matters most, since each block gets only two levels. STE wastes that flexibility: it sets s_f and b_f from the smallest and largest weight in the block, so if one weight is an outlier, one of the two levels is spent on it and everything else is squashed onto the other. And STE's gradient cannot see this, so training never fixes it. Ridge instead fits the two levels to the weights, so each level sits at the average of its group, and its gradient knows how good that fit is. That is how, at 1 bit, ridge unlocks the flexibility affine provides.